User Guide

Introduction

ItemTasker is a desktop Command Line Interface (CLI) application designed for warehouse inventory management. It lets warehouse operators efficiently track Stock Keeping Units (SKUs) and their associated maintenance or operational tasks — all from the keyboard, with no GUI overhead. Built with Java 17, ItemTasker follows a clean layered architecture (UI → Logic → Model → Storage) and models a 3×3 warehouse grid. Each SKU maps to a physical grid location (A1–C3) and carries its own task list, where every task can be assigned a due date, a priority level (HIGH / MEDIUM / LOW), and a description. The application persists all state to JSON automatically on exit and reloads it seamlessly on the next launch.

Quick Start

1. Ensure you have Java `17` or above installed on your computer.
2. Download the latest `.jar` file from here.
3. Copy the file to the folder you want to use as the home folder for ItemTasker.
4. Open a command terminal, `cd` into the folder you put the jar file in, and use the following command to run the application:

```
java -jar ItemTasker.jar
```

If the setup is correct, you should see the welcome message appear in a few seconds.

5. Type a command and press Enter to execute it. e.g. typing `help` and pressing Enter will display all available commands. Some example commands you can try:
   - `listtasks` : Lists all tasks across all SKUs.
   - `addsku n/PALLET-A l/B2` : Adds a new SKU named `PALLET-A` at warehouse location `B2`.
   - `addskutask n/PALLET-A d/2026-06-15 p/HIGH t/Check stock` : Adds a high-priority task to `PALLET-A` due on 15 June 2026.
   - `marktask n/PALLET-A i/1` : Marks the 1st task of `PALLET-A` as completed.
   - `deletesku n/PALLET-A` : Deletes `PALLET-A` and all its associated tasks.
   - `viewmap` : Displays the 3×3 warehouse grid with SKU locations.
   - `bye` / `exit` : Exits the application.
6. Refer to the Features section below for the full details of each command.

Features

Adding a SKU: `addsku`

Adds a new SKU to the warehouse at the specified grid location.

Format: `addsku n/SKU_ID l/LOCATION`

- `SKU_ID` is case-insensitive and will be stored in uppercase. It must only contain English letters, numbers, and hyphens (`-`).
- `LOCATION` must be a valid warehouse grid cell: `A1`, `A2`, `A3`, `B1`, `B2`, `B3`, `C1`, `C2`, or `C3`.
- Duplicate SKU IDs are not allowed.

Example of usage:

```
addsku n/PALLET-A l/B2
```

```
addsku n/WIDGET-X1 l/A3
```

Editing a SKU: `editsku`

Updates the warehouse location of an existing SKU.

Format: `editsku n/SKU_ID l/NEW_LOCATION`

- `SKU_ID` must already exist in the warehouse.
- `NEW_LOCATION` must be a valid warehouse grid cell.

Example of usage:

```
editsku n/PALLET-A l/C3
```

```
editsku n/WIDGET-X1 l/A1
```

Deleting a SKU: `deletesku`

Deletes an existing SKU and all its associated tasks from the warehouse.

Format: `deletesku n/SKU_ID`

- This action is irreversible — all tasks tied to the SKU will also be removed.

Example of usage:

```
deletesku n/PALLET-A
```

Adding a task: `addskutask`

Adds a new task to an existing SKU.

Format: `addskutask n/SKU_ID d/DUE_DATE [p/PRIORITY] [t/DESCRIPTION]`

- `SKU_ID` must already exist. Use `addsku` to register it first.
- `DUE_DATE` must be in `YYYY-MM-DD` format, e.g. `2026-06-15`. The year must be between `1970` and `2100`.
- Dates before today are accepted (for record-keeping of overdue work) but will print a `[WARNING]` reminder.
- `PRIORITY` is optional and must be `HIGH`, `MEDIUM`, or `LOW`. Defaults to `HIGH` if not provided.
- `DESCRIPTION` is optional. Leave it out or use an empty `t/` tag to create a task with no description.
- **Note:** Tasks with the exact same priority, due date, and description (case-insensitive) cannot be created. The app will reject identical operations to prevent duplicate tracking.

Example of usage:

```
addskutask n/PALLET-A d/2026-06-15 p/HIGH t/Check stock levels
```

```
addskutask n/WIDGET-X1 d/2026-12-01
```

Editing a task: `edittask`

Updates one or more fields of an existing task.

Format: `edittask n/SKU_ID i/TASK_INDEX [d/DUE_DATE] [p/PRIORITY] [t/DESCRIPTION]`

- At least one of `d/`, `p/`, or `t/` must be provided.
- Unlike `addskutask`, if the `t/` flag is provided, its value cannot be blank.
- `TASK_INDEX` refers to the task's position in the SKU's task list (starting from 1).
- `DUE_DATE` must be in `YYYY-MM-DD` format if provided.
- `PRIORITY` must be `HIGH`, `MEDIUM`, or `LOW` if provided.

Example of usage:

```
edittask n/PALLET-A i/1 d/2026-12-31 p/LOW
```

```
edittask n/WIDGET-X1 i/2 t/Updated description
```

Deleting a task: `deletetask`

Deletes a specific task from a SKU's task list.

Format: `deletetask n/SKU_ID i/TASK_INDEX`

- `TASK_INDEX` refers to the task's position in the SKU's task list (starting from 1).
- This action is irreversible.

Example of usage:

```
deletetask n/PALLET-A i/2
```

Marking a task: `marktask`

Marks a specific task as completed.

Format: `marktask n/SKU_ID i/TASK_INDEX`

- `TASK_INDEX` refers to the task's position in the SKU's task list (starting from 1).
- If the task is already marked as done, the application will notify you.

Example of usage:

```
marktask n/PALLET-A i/1
```

Unmarking a task: `unmarktask`

Marks a previously completed task as not done.

Format: `unmarktask n/SKU_ID i/TASK_INDEX`

- `TASK_INDEX` refers to the task's position in the SKU's task list (starting from 1).
- If the task is already unmarked, the application will notify you.

Example of usage:

```
unmarktask n/PALLET-A i/1
```

Sorting tasks: `sorttasks`

Sorts a SKU's task list by a specified field. The sort is persistent: subsequent `listtasks` calls will reflect the sorted order.

Format: `sorttasks n/SKU_ID s/SORT_FIELD [o/ORDER]`

- `SORT_FIELD` must be one of: `date`, `priority`, or `status`.
- `ORDER` is optional and must be exactly `ascending` or `descending` (no abbreviations). Defaults to `ascending` if not provided.
- For `s/date`: `ascending` shows earliest date first; `descending` shows latest date first.
- For `s/priority`: `ascending` shows HIGH → MEDIUM → LOW (highest priority first); `descending` shows LOW → MEDIUM → HIGH (lowest priority first).
- For `s/status`: `ascending` shows incomplete tasks before completed tasks; `descending` shows completed tasks first.

Example of usage:

```
sorttasks n/PALLET-A s/priority o/descending
```

```
sorttasks n/WIDGET-X1 s/date
```

Listing tasks: `listtasks`

Lists tasks across the warehouse with optional filters.

Format: `listtasks [n/SKU_ID] [p/PRIORITY] [l/LOCATION]`

- Only one filter may be used at a time.
- Using `n/` lists all tasks for a specific SKU.
- Using `p/` filters tasks by priority level (`HIGH`, `MEDIUM`, or `LOW`).
- Using `l/` lists all tasks sorted by distance from the specified grid location.
- Using no filter lists all tasks across all SKUs.

Example of usage:

```
listtasks
```

```
listtasks n/PALLET-A
```

```
listtasks p/HIGH
```

```
listtasks l/B2
```

Handling Multiple Filters:

If more than one filter is entered, the system returns an error message to indicate the command is invalid.

- **Example Command:** `listtasks n/PALLET-A p/HIGH`
- **Expected Output:** `[ERROR] Conflict: You can only use ONE filter (n/, p/, or l/) at a time.`

Finding tasks: `find`

Searches for tasks matching one or more criteria.

Format: `find [n/SKU_ID] [t/DESCRIPTION] [i/TASK_INDEX]`

- At least one filter must be provided.
- Filters can be combined. e.g. `find n/PALLET-A t/stock` returns tasks in `PALLET-A` whose description contains "stock".
- `DESCRIPTION` is a keyword match and is case-insensitive.
- `TASK_INDEX` searches for a task at that position across matching SKUs.

Example of usage:

```
find n/PALLET-A t/stock
```

```
find t/check
```

```
find n/PALLET-A i/1
```

Viewing warehouse map: `viewmap`

Displays a visual representation of the 3×3 warehouse grid.

Map Logic:

- **What the numbers mean:** Each cell (e.g., `[B2: 05]`) displays the **total number of tasks** (both completed and incomplete) assigned to all SKUs at that location.
- **Why a location might show `00`:** If a location shows `00`, it means there are no tasks created for the SKUs at that spot. It does **not** mean the location is empty of SKUs.

Example of usage:

```
viewmap
```

Checking status: `status`

Displays completion statistics for one or all SKUs.

Format: `status [n/SKU_ID]`

- Using no filter shows completion statistics for all SKUs in the warehouse.
- Using `n/` shows statistics for a specific SKU only.

Example of usage:

```
status
```

```
status n/PALLET-A
```

Exporting inventory: `export`

Saves a human-readable snapshot of the full warehouse inventory to a text file.

Format: `export`

Example of usage:

```
export
```

Viewing help: `help`

Displays a summary of all available commands.

Format: `help`

Example of usage:

```
help
```

Exiting the application: `bye` / `exit`

Exits ItemTasker. All data is automatically saved before the application closes. Both `bye` and `exit` are accepted and behave identically.

Format: `bye` or `exit`

Example of usage:

```
bye
```

```
exit
```

FAQ

Q: How do I transfer my warehouse data to another computer?
A: Install the application on the other computer and run it once to generate the default folders. Then, simply overwrite the `Data/storage.json` file it creates with the `storage.json` file from your previous computer.

Q: Do I need to manually save my tasks before closing the application?
A: No. ItemTasker automatically saves your entire inventory and task list to the hard disk whenever you close the application using the `bye` or `exit` commands. Just ensure you exit the app properly instead of force-closing the terminal!

Q: Can I use my own custom location names like "Loading-Dock" or "Aisle-12"?
A: Currently, ItemTasker strictly uses a standardized 3x3 grid system (A1 through C3) to ensure spatial sorting and distance calculations work instantly. You must assign SKUs to one of the 9 predefined sectors.

Q: How does the `listtasks l/LOCATION` command calculate distance?
A: It calculates the "Manhattan Distance" across the warehouse grid. It measures the physical grid steps required to move from your specified location to the SKU's location, bringing the closest tasks to the top of your list so you can clear them efficiently.

Q: What happens if I manually edit the `storage.json` file and make a mistake?
A: If the JSON format becomes invalid, outdated, or corrupted due to manual edits, ItemTasker will print a warning on startup and begin with an empty warehouse to prevent system crashes. It is highly recommended to make a copy of your `storage.json` file before doing any manual tweaking.

Q: What is the difference between `viewmap` and `status`?
A: `viewmap` is a visual tool that shows the physical distribution of tasks across the A1-C3 grid. `status` is an analytical tool that provides a breakdown of task completion percentages and identifies which SKUs require immediate attention.

Q: Where can I find the application logs if I need to report a bug?
A: ItemTasker automatically generates an `itemtasker.log` file in the same directory as the application containing diagnostic information. You can review or attach this file when reporting any unexpected behavior.

Q: Is there a limit to how many SKUs or tasks I can add?
A: There is no hardcoded limit to the number of SKUs or tasks you can manage. However, performance might begin to slow down if you are managing tens of thousands of tasks simultaneously on older hardware.

Q: Can I edit multiple fields of a task at once?
A: Yes. The `edittask` command accepts any combination of `d/` (due date), `p/` (priority), and `t/` (description) flags simultaneously. Only the fields you specify will be updated. All other fields are preserved.

Q: Can I have more than one SKU in the same grid square (e.g., A1)?
A: Yes. ItemTasker treats grid locations as "sectors." You can store multiple different SKUs in the same sector. The `viewmap` will show the sum of all tasks for all SKUs in that specific sector.

Command Summary

Action	Format	Example
Add SKU	`addsku n/SKU_ID l/LOCATION`	`addsku n/PALLET-A l/B2`
Edit SKU	`editsku n/SKU_ID l/NEW_LOCATION`	`editsku n/PALLET-A l/C3`
Delete SKU	`deletesku n/SKU_ID`	`deletesku n/PALLET-A`
Add task	`addskutask n/SKU_ID d/DUE_DATE [p/PRIORITY] [t/DESCRIPTION]`	`addskutask n/PALLET-A d/2026-06-15 p/HIGH t/Check stock`
Edit task	`edittask n/SKU_ID i/TASK_INDEX [d/DUE_DATE] [p/PRIORITY] [t/DESCRIPTION]`	`edittask n/PALLET-A i/1 d/2026-12-31 p/LOW`
Delete task	`deletetask n/SKU_ID i/TASK_INDEX`	`deletetask n/PALLET-A i/2`
Mark task	`marktask n/SKU_ID i/TASK_INDEX`	`marktask n/PALLET-A i/1`
Unmark task	`unmarktask n/SKU_ID i/TASK_INDEX`	`unmarktask n/PALLET-A i/1`
Sort tasks	`sorttasks n/SKU_ID s/SORT_FIELD [o/ORDER]`	`sorttasks n/PALLET-A s/priority o/descending`
List all tasks	`listtasks`	`listtasks`
List by SKU	`listtasks n/SKU_ID`	`listtasks n/PALLET-A`
List by priority	`listtasks p/PRIORITY`	`listtasks p/HIGH`
List by distance	`listtasks l/LOCATION`	`listtasks l/B2`
Find tasks	`find [n/SKU_ID] [t/DESCRIPTION] [i/TASK_INDEX]`	`find n/PALLET-A t/Check`
View warehouse map	`viewmap`	`viewmap`
Export inventory	`export`	`export`
View status	`status [n/SKU_ID]`	`status n/PALLET-A`
Help	`help`	`help`
Exit	`bye` / `exit`	`bye`

Locations: A1 A2 A3 | B1 B2 B3 | C1 C2 C3 (3x3 warehouse grid). Priorities: HIGH, MEDIUM, LOW. Sort fields: date, priority, status.